

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.15

INDEXING

 Copy as Markdown

Indexing

Queries that [filter](#) or [sort](#) a large table slow down as rows accumulate, because the database has to scan every row. An index lets the database look up the matching rows directly instead. You declare indexes in the model file, and the next [migration](#) creates them in the database.

Add an index

To add an index, add an `indexes` section to the model file. The `indexes` section is a map where the key is the name of the index and the value is a map with the index details.

```
class: Company
table: company
fields:
  name: String
indexes:
  company_name_idx:
    fields: name
```

The `fields` keyword holds a comma-separated list of column names. These are the fields upon which the index is created. Note that the index can contain several fields.

```
class: Company
table: company
fields:
  name: String
  foundedAt: DateTime
indexes:
  company_idx:
    fields: name, foundedAt
```



Ask AI

Make fields unique

A unique index ensures that no two rows store the same value in the indexed fields. This is useful for example if you want to make sure that no two companies have the same name. For a single field, mark it with the `unique` keyword directly on the field definition:

```
class: Company
table: company
fields:
  name: String, unique
```

To make a combination of fields unique, or to name the index yourself, declare the index in the `indexes` section and set `unique: true`:

```
class: Company
table: company
fields:
  name: String
indexes:
  company_name_idx:
    fields: name
    unique: true
```

The `unique` keyword on an index is a bool that defaults to false. When it is applied to a multi-column index, the combination of the fields must be unique.

Composite unique constraints

When a value should be unique only within a scope (for example, a setting key that is unique per user), use `unique(per=...)` on the field that must be unique within that scope. Serverpod auto-generates a composite unique index with the `per` columns first, followed by the annotated field.

```
class: UserSetting
table: user_setting
fields:
  userId: int
  key: String, unique(per=userId)
```

In this example, two rows can share the same `key` if they belong to different users, but the same user cannot have two rows with the same `key`.

For a scope that spans multiple columns, pass a list of field names:

```
class: Product
table: product
fields:
  tenantId: int
  category: String
```

```
sku: String, unique(per=[tenantId, category])
```

You can also use the equivalent expanded form, where you name the index yourself:

```
class: Product
table: product
fields:
  tenantId: int
  category: String
  sku: String
indexes:
  product_unique_idx:
    fields: tenantId, category, sku
    unique: true
```

Null values in unique indexes

A unique index treats null values as distinct, so any number of rows can hold null in an indexed field without colliding. Set `nulls_distinct: false` to make them collide instead, which is what turns a nullable column into a "one row per key" constraint:

```
class: Product
table: product
fields:
  tenantId: int
  sku: String
  deletedAt: DateTime?
indexes:
  product_active_sku_idx:
    fields: tenantId, sku, deletedAt
    unique: true
    nulls_distinct: false
```

In this example, a tenant can have at most one product with a given `sku` and no `deletedAt`. Deleted rows carry a timestamp, so they no longer collide with each other.

The key is only allowed on unique indexes, and leaving it out keeps the database default. SQLite cannot express `nulls_distinct: false`, so creating a SQLite migration for a model that sets it fails. Setting it to `true` is accepted, since that matches what SQLite already does.

Specify an index type

Add a `type` key to specify the index type.

```
class: Company
table: company
fields:
```

```

name: String
indexes:
  company_name_idx:
    fields: name
    type: brin

```

If no type is specified, the default is `btree`. All Postgres index types are supported: `btree`, `hash`, `gist`, `spgist`, `gin`, and `brin`.

ⓘ INFO

Index types other than `btree` are Postgres-only, including the GIN, vector, and geography indexes described below. On SQLite, indexes declared with other types are skipped when a migration is created, and a warning is logged.

GIN indexes

GIN (Generalized Inverted Index) indexes are designed for efficiently querying composite values such as JSONB data. When all fields in an index are stored as `jsonb`, Serverpod automatically defaults the index type to `gin`:

```

class: Product
table: product
fields:
  tags: List<String>, serializationDataType=jsonb
indexes:
  product_tags_idx:
    fields: tags
    # type defaults to gin since all indexed fields are jsonb

```

You can also set the type explicitly:

```

indexes:
  product_tags_idx:
    fields: tags
    type: gin

```

Operator classes

GIN indexes support different operator classes that control which query operators the index can accelerate. Use the `operatorClass` keyword to specify one:

```

indexes:
  product_tags_idx:
    fields: tags

```

```
type: gin
operatorClass: jsonbPathOps
```

Operator Class	Description	Use Case
<code>jsonbOps</code>	Default. Supports <code>@></code> , <code>?</code> , <code>? </code> , <code>?&</code> operators	General-purpose JSONB querying
<code>jsonbPathOps</code>	Supports only <code>@></code> (containment)	Faster and smaller index when you only need containment queries
<code>arrayOps</code>	For array containment queries	Array-typed columns
<code>tsvectorOps</code>	For full-text search	Text search with <code>tsvector</code> columns

TIP

If you only need containment queries (`@>`), use `jsonbPathOps`: it produces a smaller and faster index than the default `jsonbOps`.

For details on configuring JSONB storage on your model fields, see [Storing serializable fields as JSONB](#).

Vector indexes

Vector similarity searches benefit from specialized indexes on vector fields (`Vector`, `HalfVector`, `SparseVector`, `Bit`). Serverpod supports the `hnsw` and `ivfflat` index types.

INFO

Each vector index can only be created on a single vector field. It is not possible to create a vector index on multiple fields of any kind.

HNSW indexes

Hierarchical Navigable Small World (HNSW) indexes provide fast approximate nearest neighbor search:

```
class: Document
table: document
fields:
  content: String
```

```

embedding: Vector(1536)
keywords: SparseVector(10000)
hash: Bit(256)
indexes:
  document_embedding_hnsw_idx:
    fields: embedding
    type: hnsw
    distanceFunction: cosine
    parameters:
      m: 16
      ef_construction: 64
  document_keywords_idx:
    fields: keywords
    type: hnsw
    distanceFunction: innerProduct
    parameters:
      m: 16
      ef_construction: 64
  document_hash_idx:
    fields: hash
    type: hnsw
    distanceFunction: hamming
    parameters:
      m: 16
      ef_construction: 64

```

Available HNSW parameters:

- **m**: Maximum number of bidirectional links for each node (default: 16)
- **ef_construction**: Size of the dynamic candidate list (default: 64)

Serverpod validates that **ef_construction** is at least $2 * m$, and rejects the model file otherwise.

IVFFLAT indexes

Inverted File with Flat compression (IVFFLAT) indexes are suitable for large datasets:

```

class: Document
table: document
fields:
  content: String
  embedding: Vector(1536)
indexes:
  document_embedding_ivfflat_idx:
    fields: embedding
    type: ivfflat
    distanceFunction: innerProduct
    parameters:
      lists: 100

```

Available IVFFLAT parameters:

- `lists`: Number of inverted lists (default: 100)

Distance functions

Supported distance functions for vector indexes (`distanceFunction` parameter):

Distance Function	Description	Use Case
<code>l2</code>	Euclidean distance	Default for most embeddings
<code>innerProduct</code>	Inner product	When vectors are normalized
<code>cosine</code>	Cosine distance	Text embeddings
<code>l1</code>	Manhattan or taxicab distance	Sparse/high-dimensional data
<code>hamming</code>	Hamming distance	Binary vectors (Bit type)
<code>jaccard</code>	Jaccard distance	Binary vectors (Bit type)

Different vector types have specific limitations when creating indexes:

- **SparseVector**: Can only use HNSW indexes (IVFFLAT is not supported).
- **HalfVector**: When using IVFFLAT indexes, the L1 distance function is not supported.
- **Bit**: Only supports `hamming` (default) and `jaccard` distance functions.

TIP

If more than one distance function is going to be frequently used on the same vector field, consider creating one index for each distance function to ensure optimal performance.

For more details on vector indexes and their configuration, refer to the [pgvector extension documentation](#).

Geography indexes

Geography columns benefit from spatial indexes, which significantly improve the performance of spatial queries such as proximity searches, intersection tests, and containment checks. Two index types are available for geography fields:

- `gist` - Generalized Search Tree, the default and the right choice for most workloads.
- `spgist` - Space-Partitioned Generalized Search Tree.

If no `type` is specified for an index on a geography field, it defaults to `gist`.

```
class: Store
table: store
fields:
  name: String
  location: GeographyPoint
indexes:
  store_location_idx:
    fields: location
    type: gist
```

Use `spgist` by setting the index `type` explicitly:

```
class: DeliveryZone
table: delivery_zone
fields:
  name: String
  boundary: GeographyPolygon
indexes:
  delivery_zone_boundary_idx:
    fields: boundary
    type: spgist
```

TIP

A spatial index accelerates all spatial operations (`intersects`, `distanceWithin`, `distance`, `contains`, `within`). For tables with many rows and frequent spatial queries, adding one is strongly recommended.

Two restrictions apply to geography indexes:

- Geography fields only support the `gist` and `spgist` index types. Specifying any other type fails code generation with `The "type" property must be one of: gist, spgist.`
- An index may cover several geography columns, but it cannot mix geography and non-geography fields. Doing so fails with `Mixing geography and non-geography fields in the same index is not allowed.`

INFO

Indexes of type `spgist` on the geography type require a recent version of PostGIS. If your PostgreSQL instance ships an older PostGIS, use `gist` instead.